Environment-Aware, Label-Efficient WiFi CSI Activity Recognition: Cross-Room Few-Shot Adaptation

CS60055 Ubiquitous Computing | Hackathon Challenge 2 | Technical Report

Harshit Singhal (Roll No. 23CS10025) — sole team member

Repository: https://github.com/harshitsh1/csi_hackathon

1. Introduction

A WiFi sensing model trained in one room tends to absorb room-specific information (multipath geometry, gain state, session conditions) along with activity information, and a deployment in a new room can afford only a few labelled examples. This report studies that setting on the Frankfurt CSI Human Activity dataset (Schäfer et al., Applied Sciences 2021) with two recorded environments: Room A (Experiment 2, raw Nexmon PCAP, BCM4366c0, 80 MHz) and Room B (Experiment 3, amplitude CSV). The central question is label efficiency: how does macro-F1 change as the number of labelled target captures K goes from 1 to 5 to 10 to 20, and how much do self-supervised pretraining and room routing contribute?

The submission is a complete Python pipeline: PCAP parsing and Nexmon packed-CSI decoding written from public Nexmon documentation, quality control, a common 256-column intermediate format, deterministic windowing, development-only normalisation, a room-fingerprint router (Task 1), a contrastive 1-D CNN encoder (Task 2), few-label cross-room heads (Task 3), and a single inference path shared by the validation export and run_episode.py. All numbers below are validation results on the fixed validation sets, not unseen-test accuracy.

2. Data and Splits

Room A holds 1,085 raw PCAP captures in class folders (fall, sit, sitdown, stand, standup, walk) plus the supplied validation/ folder (25 per class, 150 captures). The remaining 935 are development captures (the specification lists 934; our count is one higher). Room B holds 1,201 amplitude CSVs labelled by filename prefix (Empty, Lying, Sitting, Standing, Walking); every file has 500 rows and 256 columns. Labels come from folder or file names and are used only as supervision targets.

Room B has no validation folder, so 25 captures per class were drawn at capture level: candidate names are sorted, and for each class random.Random(42).sample(names, 25) is taken (125 validation, 1,076 development captures). The capture is the atomic unit: every window inherits its capture's split, and no capture is shared between development and validation. Raw files are never modified; the split is recorded in manifest.csv.

Table 1. Captures and windows per split

Room	Split	Captures	Windows	Notes
A	development	935	1,179	stride 75; 9 captures < 150 valid rows skipped
A	validation	150	147	stride 150; 3 captures < 150 valid rows skipped
B	development	1,076	5,380	stride 75; 5 windows per capture
B	validation	125	375	stride 150; 3 windows per capture
Total		2,286	7,081	6,559 development / 522 validation windows

Inspection of Room A found that every frame is valid UDP carrying the Nexmon magic 0x1111 and 256 words (0 non-UDP, 0 non-Nexmon, 0 truncated, 0 wrong-length). Frames alternate between full-band frames and narrow-band frames with sequence field 0xFFFF: 118,013 of 234,239 frames (50.4%) are of the latter kind, with energy confined to roughly the central 64 subcarriers. Captures are on average 215.9 valid frames long (minimum 112, maximum 251).

3. Preprocessing Pipeline

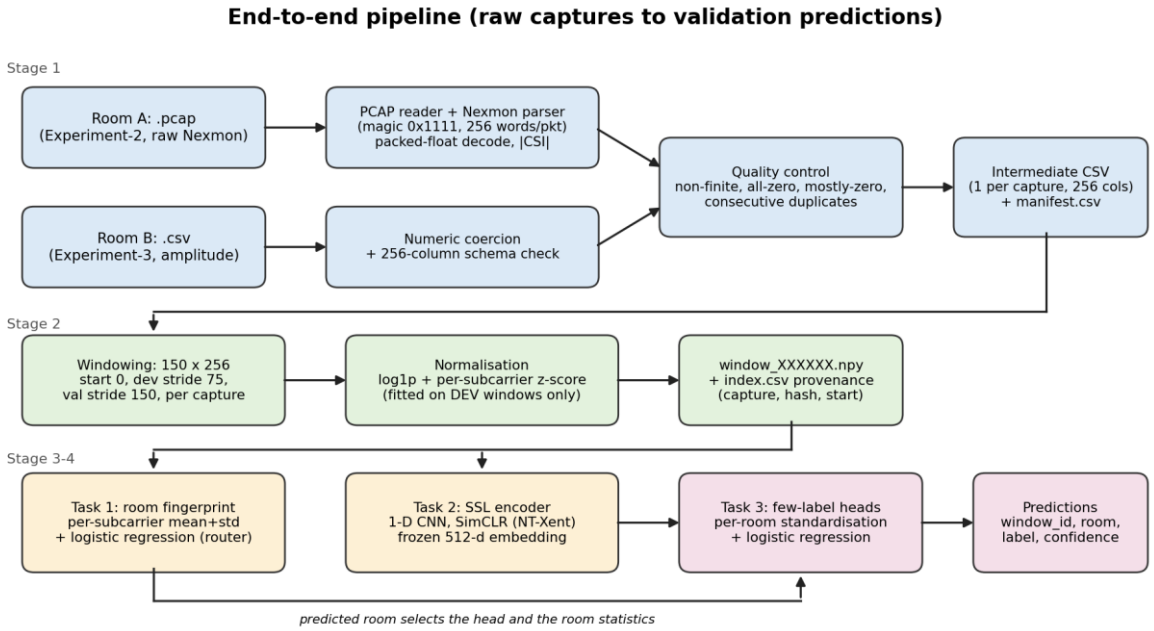


Figure 1. End-to-end pipeline from raw captures to validation predictions.

3.1 Room A: PCAP to amplitude

A minimal classic-PCAP reader (struct only) handles microsecond and nanosecond magic numbers in either byte order, counts truncated trailing records and rejects pcapng. Ethernet/IPv4/UDP payloads are extracted (802.1Q tags allowed). A payload is Nexmon CSI when its first two bytes are 0x1111; the 18-byte header holds RSSI, frame control, MAC, sequence number, core/stream word, chanspec and chip id, and the remaining bytes are little-endian uint32 words. Packets with other than 256 words are discarded and counted.

Each word is a BCM4366c0 packed float: bits [5:0] hold a 6-bit signed exponent, bits [16:6] the imaginary mantissa magnitude, bits [28:18] the real mantissa magnitude, and the signs sit at bit 29 (real) and bit 17 (imaginary). The value is sign × mantissa × 2^exponent, and all-zero words decode to 0. With autoscale on, each packet receives one common exponent shift so that its largest component has about 10 bits; this is a per-packet constant and does not change relative subcarrier amplitudes. Amplitude is |CSI|, and temporal packet order is preserved.

Stream policy: we keep all packets (stream_policy = all). Full-band packets alone are too few per capture (about 108) to form contiguous 150-observation windows, so dropping the 0xFFFF frames would remove most captures. The consequence is that Room A windows interleave full-band and narrow-band observations, a property Room B lacks (see Section 6.1).

3.2 Room B and subcarrier order

Room B CSVs are read with pandas, coerced to numeric (unparsable cells become NaN for QC) and checked for exactly 256 columns. Spectral inspection of Room A showed the DC null already at index ≈128, matching the Room B column layout, so no fftshift is applied (fftshift: false) and both rooms share the same ascending-frequency order.

3.3 Quality control

Row-wise, order-preserving checks: non-finite rows, all-zero rows, mostly-zero rows (more than 50% zero subcarriers) and consecutive exact duplicates are removed, and counts are logged per capture. On this dataset all four counters were zero for both rooms (2,286 captures processed, 2,286 kept), so QC changed no rows; the only loss comes from captures shorter than 150 rows (Table 1). No denoising was applied, since the supplied hints do not require it and filtering could remove the fast Doppler content that distinguishes motions.

3.4 Windowing, normalisation and provenance

A window is 150 consecutive post-QC observations × 256 subcarriers, starting at observation 0 with stride 75 for development (50% overlap, more training data) and 150 for validation (non-overlapping), never crossing a capture boundary. At inference a capture shorter than 150 rows is padded by repeating its last valid row (disable with --no_pad).

Amplitudes are transformed by log1p and then z-scored per subcarrier with a floor of 0.05 on the standard deviation. The mean and standard deviation are fitted once on development windows pooled over both rooms (no labels), saved in normalizer.npz and frozen; validation and episode data are only transformed. Every window has a row in windows/index.csv with window_id, split, capture_id, room, label, start, length, stride, source file and a preprocessing hash (SHA-1 over the decode, qc, split, window and normalization config sections; value 16cb948fb2), so each window is traceable to its capture and exact configuration.

4. Methods

4.1 Task 1: recorded-environment fingerprint

The fingerprint is the static channel: per-subcarrier temporal mean and standard deviation over the first L observations of a normalised window (512 features), followed by StandardScaler and multinomial logistic regression (C = 1). It is trained on development windows cropped to $L \in \{10, 25, 50, 100, 150\}$, so one classifier serves all lengths. Only the numeric window is used; no filename, extension, capture length or packet count. The same classifier is the router of the full system.

4.2 Task 2: self-supervised encoder

The encoder treats subcarriers as channels of a 1-D CNN over time: Conv1d(256→192, k5, s2), Conv1d(192→256, k5, s2), Conv1d(256→256, k3), each followed by BatchNorm and GELU. Mean and standard-deviation pooling over time gives a 512-d feature; a projection head (512→256→128, L2-normalised) is used only for pretraining. The objective is SimCLR-style NT-Xent (temperature 0.2) between two label-free views of each window. Augmentations: random temporal crop (60–100% of the window, resized to 150), global gain jitter (σ 0.3), per-subcarrier shift (σ 0.1), Gaussian noise (σ 0.05), subcarrier dropout (p 0.15) and a random time mask (up to 20 steps). Training uses AdamW (lr 1e-3, weight decay 1e-4), batch 128, 30 epochs with cosine decay, on development windows of both rooms, with no activity or room labels.

Evaluation freezes the encoder, standardises the 512-d features and fits a class-balanced logistic regression on labelled development windows over the unified 8-label space. Baselines: the same architecture with random weights (seed 0) and 128 handcrafted features (per 16-subcarrier-band mean, std, mean absolute difference, range, and four log spectral band energies).

4.3 Task 3: cross-room few-label adaptation

For target room T and source room S, the shared classes SIT, STAND and WALK are learned only from S's labelled development windows; the target-exclusive classes (A: SITDOWN, STANDUP, FALL; B: EMPTY, LYING) are learned from K distinct development captures of T per class, sampled without replacement (all windows of a capture count as one shot). Each room's embeddings are z-scored with the mean and standard deviation of that room's unlabelled development embeddings (unsupervised alignment). The head is a class-balanced logistic regression (C = 1) on the union of standardised source windows and standardised support windows. Each K is repeated for 5 random support draws and reported as mean ± std. An assertion checks that all support captures are development captures of the target room, so validation can never enter support.

4.4 Routing and leakage controls

The full system routes each window with the Task 1 classifier to the head and room statistics of the predicted room. Three modes are compared on pooled validation windows in the 8-label space:

predicted-room routing, oracle-room (true room selects the head), and no routing (both heads run on every window and the more confident prediction wins). Leakage controls: capture-level splits; normaliser, encoder, router, room statistics and heads fitted on development data only; support sets from target development captures only; models receive only numeric (150, 256) arrays.

5. Experimental Setup

Hardware: Intel Core i7 (12 logical cores), NVIDIA RTX 4050 Laptop GPU (6 GB), 16 GB RAM; Windows 11, Python 3.12, PyTorch 2.5.1 (CUDA 12.1), scikit-learn 1.6.1, NumPy 2.2.3, pandas 2.2.3. Global seed 42. Approximate run times: ingestion of 2,286 captures 78 s (4 workers), windowing 12 s, contrastive pretraining 118 s (GPU), tasks and figures 55 s.

```
python run_pipeline.py --config configs/experiment.yaml --stage all --workers 4 --device cuda
python run_episode.py --episode_dir <episode> --out predictions.csv [--support_csv support.csv]
```

6. Results (validation sets)

6.1 Task 1: environment fingerprint

Room accuracy is 1.000 on the 522 validation windows (Room A 1.000, Room B 1.000) at every tested length L ∈ {10, 25, 50, 100, 150}, so the headline L = 150 number is 100%.

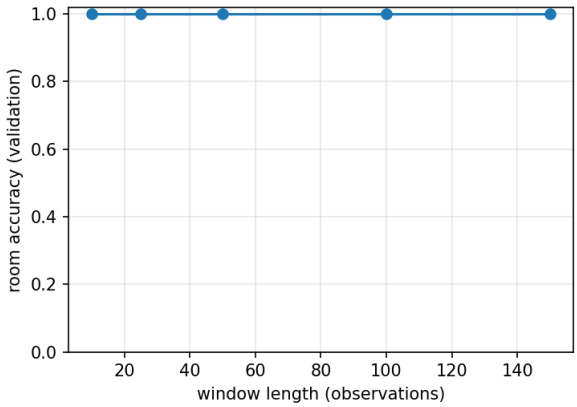


Figure 2. Room accuracy against window length (validation).

Perfect accuracy even at 10 observations should not be read as evidence of robust room recognition. Rooms A and B were recorded in different sessions, so the classifier can use session-specific effects (gain state, offsets) as well as geometry. A second possible cue, which we did not test, is that Room A windows interleave full-band and narrow-band frames while Room B windows do not. The router is therefore a recorded-environment identifier, as the task describes it, and not a general room recogniser.

6.2 Task 2: frozen-representation linear probe

Table 2. Linear probe on frozen features (8-label space; validation)

Representation	Macro-F1	Balanced acc.	Macro-F1 Room A	Macro-F1 Room B
SSL encoder	0.951	0.959	0.910	0.989
Random encoder	0.924	0.926	0.941	0.885
Handcrafted	0.893	0.929	0.778	0.951

The SSL encoder has the highest pooled macro-F1 (0.951 versus 0.924 random and 0.893 handcrafted) and the highest Room B score, but the margin over the random-weight network is only 0.028, and in Room A the random encoder is better (0.941 versus 0.910). This is a single training run, so we regard the evidence for SSL over a random convolutional feature extractor as

modest; the clearer gain is over the handcrafted features, especially in Room A (0.910 versus 0.778).

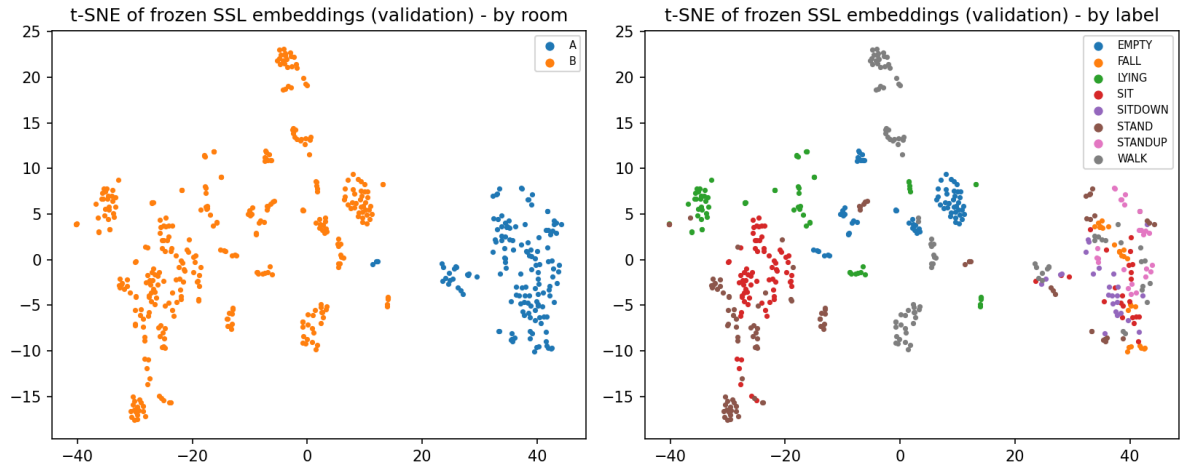


Figure 3. t-SNE of frozen SSL embeddings (validation), coloured by room (left) and by activity (right).

In the embedding the two rooms form fully separate groups, which agrees with Task 1 and means the encoder keeps room identity. Within Room B, EMPTY, LYING and WALK form compact clusters; within Room A, SIT, STAND, SITDOWN, STANDUP and FALL overlap heavily in one region, matching the weaker Room A results above and in Task 3.

6.3 Task 3: cross-room few-label adaptation

Table 3. Target Room A (source B), SSL encoder, mean ± std over 5 draws

K	Macro-F1	Target-exclusive F1	Shared F1	Balanced acc.
1	0.228 ± 0.026	0.337 ± 0.062	0.119 ± 0.037	0.261 ± 0.018
5	0.305 ± 0.028	0.492 ± 0.080	0.118 ± 0.037	0.359 ± 0.040
10	0.345 ± 0.021	0.599 ± 0.023	0.091 ± 0.042	0.432 ± 0.022
20	0.375 ± 0.021	0.658 ± 0.019	0.091 ± 0.028	0.479 ± 0.017

Table 4. Target Room B (source A), SSL encoder, mean ± std over 5 draws

K	Macro-F1	Target-exclusive F1	Shared F1	Balanced acc.
1	0.246 ± 0.049	0.292 ± 0.112	0.215 ± 0.012	0.249 ± 0.046
5	0.400 ± 0.022	0.679 ± 0.031	0.214 ± 0.027	0.403 ± 0.038
10	0.406 ± 0.024	0.754 ± 0.028	0.174 ± 0.023	0.433 ± 0.025
20	0.429 ± 0.037	0.787 ± 0.030	0.190 ± 0.047	0.466 ± 0.040

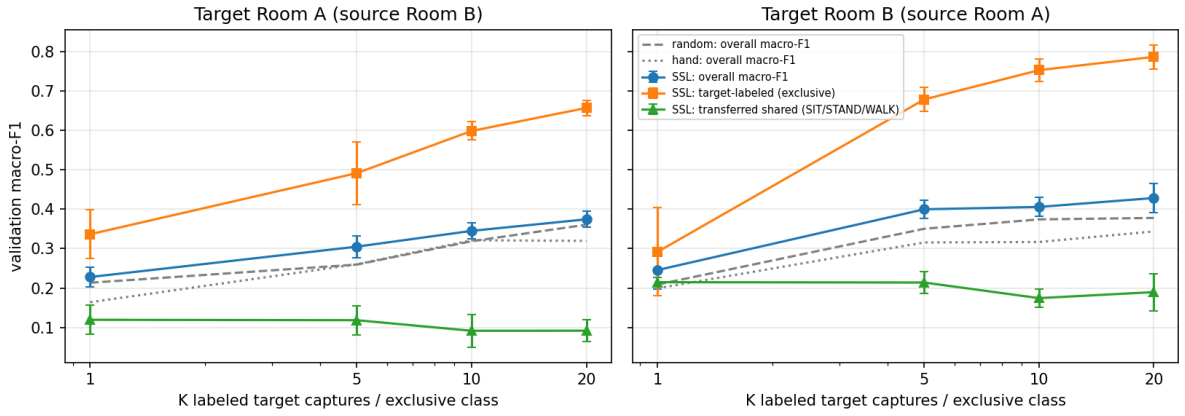


Figure 4. Label-efficiency curves: overall, target-exclusive and transferred shared macro-F1 against K, with random-encoder and handcrafted baselines.

Table 5. Overall / target-exclusive / shared macro-F1 by representation (S = SSL, R = random, H = handcrafted)

Room	K	All S	All R	All H	Excl S	Excl R	Excl H	Shared S	Shared R	Shared H
A	1	0.228	0.213	0.164	0.337	0.322	0.251	0.119	0.105	0.077
A	5	0.305	0.260	0.260	0.492	0.445	0.454	0.118	0.074	0.067
A	10	0.345	0.319	0.321	0.599	0.593	0.598	0.091	0.045	0.045
A	20	0.375	0.361	0.320	0.658	0.667	0.616	0.091	0.055	0.023
B	1	0.246	0.209	0.199	0.292	0.278	0.402	0.215	0.163	0.064
B	5	0.400	0.351	0.316	0.679	0.608	0.589	0.214	0.179	0.134
B	10	0.406	0.374	0.317	0.754	0.671	0.623	0.174	0.177	0.113
B	20	0.429	0.378	0.344	0.787	0.666	0.689	0.190	0.186	0.114

Target-exclusive activities are learned quickly and keep improving with K: in Room A target-exclusive F1 rises from 0.337 (K=1) to 0.658 (K=20), in Room B from 0.292 to 0.787. Overall macro-F1 at K=20 is 0.375 (A) and 0.429 (B). Against the baselines, SSL has the highest overall macro-F1 at every K and in both rooms, but several gaps are within one standard deviation (for example Room A, K=20: 0.375 ± 0.021 versus 0.361 ± 0.023 for random), and on target-exclusive F1 in Room A at K=20 the random encoder is marginally higher (0.667 versus 0.658). The advantage of SSL is clearest in Room B (target-exclusive 0.787 versus 0.666 random and 0.689 handcrafted at K=20).

The main weakness is transfer of the shared activities. Shared macro-F1 stays between 0.09 and 0.12 in Room A and between 0.17 and 0.21 in Room B, does not improve with K (it drifts down in Room A), and is only moderately higher than the baselines. In other words, a SIT/STAND/WALK head trained in the source room largely fails to recognise those activities in the target room, and the overall macro-F1 is held down by this. The confusion matrices below show the failure mode.

6.4 Routing contribution

Table 6. Pooled 8-label validation metrics by routing mode

K	F1 none	F1 predicted	F1 oracle	Acc none	Acc routed	Bal. acc none	Bal. acc routed
1	0.200	0.274	0.274	0.386	0.253	0.260	0.296
5	0.283	0.430	0.430	0.433	0.392	0.316	0.470
10	0.295	0.475	0.475	0.440	0.435	0.318	0.559
20	0.333	0.511	0.511	0.453	0.472	0.347	0.612

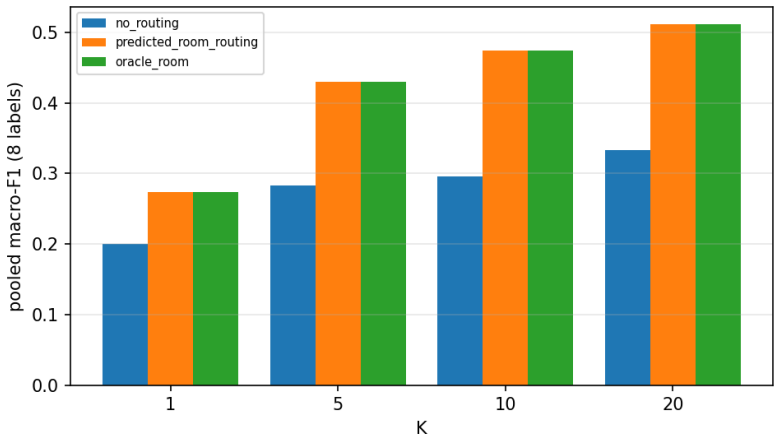


Figure 5. Pooled macro-F1 for no routing, predicted-room routing and oracle room.

Because the router is 100% accurate on validation, predicted-room routing is identical to oracle routing at every K. Routing raises pooled macro-F1 over the no-routing variant by 0.074, 0.147,

0.180 and 0.178 at K = 1, 5, 10, 20 (0.274 to 0.511 versus 0.200 to 0.333), and balanced accuracy by a similar amount. Raw accuracy tells a different story: the unrouted variant is higher at K = 1, 5 and 10 (for example 0.386 versus 0.253 at K=1) and lower only at K=20 (0.453 versus 0.472). We read this as routing improving per-class balance, not overall hit rate: the confidence-based winner without routing seems to favour the more frequent or more confident classes, but we did not analyse this further. Note also that the per-head results in Section 6.3 use the true room for each window, so they match the oracle row.

6.5 Confusion matrices at K = 5

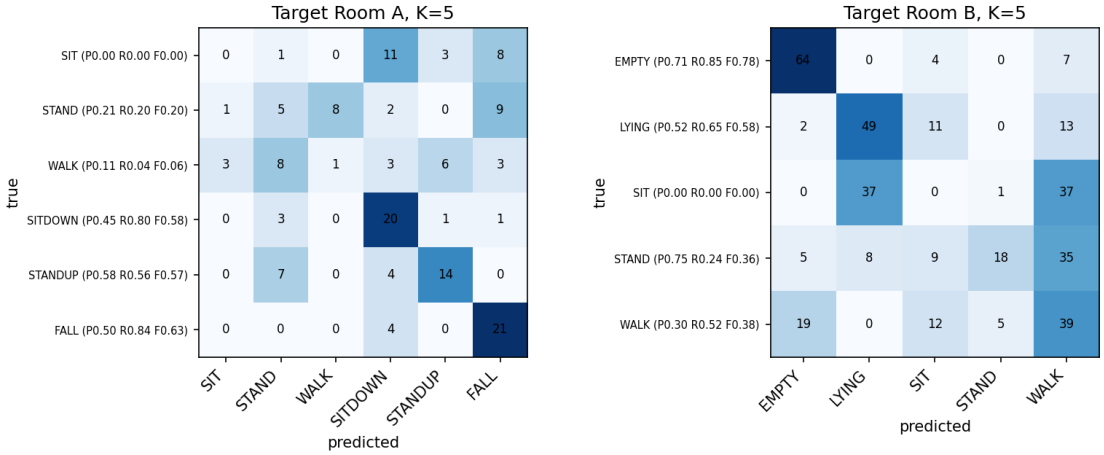


Figure 6. Confusion matrices at K = 5 for target Room A (left) and Room B (right), with per-activity precision (P), recall (R) and F1 (F) in the row labels.

Room A (first draw): FALL (F1 0.627, recall 0.84), SITDOWN (0.580, recall 0.80) and STANDUP (0.571) are recognised reasonably well, which is consistent with their distinct motion signatures. SIT, STAND and WALK, which were never labelled in the target room, score F1 of 0.000, 0.204 and 0.061; their windows are mostly assigned to the labelled target classes (for example 11 of 23 SIT windows to SITDOWN and 8 to FALL, 9 of 25 STAND windows to FALL). Room B: EMPTY (F1 0.776) and LYING (0.580) are the best classes; SIT has F1 0.000 (37 windows to LYING and 37 to WALK), STAND has precision 0.75 but recall only 0.24 (35 of 75 to WALK), and WALK has F1 0.379. In both rooms the target-exclusive classes absorb windows of the shared activities, and the SIT-like static postures are confused with their dynamic or lying neighbours, which is plausible physically since static postures carry little Doppler energy. Room A also has an activity/location confound (Section 8).

6.6 What was and was not ablated

Task 1 was evaluated at five window lengths (Figure 2), and Tasks 2 and 3 compare SSL, random-weight and handcrafted representations. We did not run separate ablations of the per-room embedding standardisation, of the head type (a prototype head exists in the code but the reported results use logistic regression), or of individual augmentations, so we make no quantitative claim about their contribution.

7. Discussion

- **What worked.** The decoding, QC and windowing pipeline produced clean, auditable windows; room identification is solved on this data; target-exclusive activities become recognisable with few captures (F1 0.66 to 0.79 at K=20); and room routing clearly improves macro-F1 and balanced accuracy of the combined system.
- **What did not.** Zero-label transfer of SIT, STAND and WALK across rooms is poor and does not improve with more target labels, so the headline overall macro-F1 stays at 0.37 to 0.43. Per-room embedding standardisation removes first- and second-order shifts but evidently not the differences in how these activities appear in each room. A plausible contributor is that

the head sees target-exclusive classes with target statistics and shared classes only with source statistics; this is a hypothesis we did not test.

- **Self-supervision.** Contrastive pretraining helps over handcrafted features and modestly over random weights, but a random convolutional encoder is already a strong baseline here, which limits what can be attributed to the SSL objective in this experiment.
- **Next steps.** Test the interleaved-frame cue in Task 1 by training on full-band frames only; add target unlabelled data for shared-class alignment (for example per-class prototypes or test-time adaptation); run multiple SSL seeds; and ablate standardisation and head types.

8. Limitations

1. Room versus session confound: the rooms were recorded in separate sessions, so the router and encoder may exploit session effects as well as physical environment.
2. Activity versus location confound: in Experiment 2 some activities were performed in different spatial zones, so activity and location are correlated.
3. Only two core environments: the results do not speak to a completely unseen room.
4. The split is capture-disjoint, not subject-disjoint.
5. Results are not comparable with the original paper's random 80/20 split.
6. Validation was used for design decisions, so the numbers are validation results, not unseen-test accuracy.
7. The SSL-versus-random comparison rests on one training run; Task 3 variability covers support draws only.
8. Short captures are padded by repeating the last row at inference; 12 Room A captures (9 development, 3 validation) with fewer than 150 valid rows produce no windows in the reported experiments.

9. Reproducibility

The repository (https://github.com/harshitsh1/csi_hackathon) contains the source under src/, the configuration configs/experiment.yaml, run_pipeline.py (stages ingest, windows, train, tasks) and run_episode.py. Raw data is not committed. The pipeline regenerates intermediate CSVs, windows, normaliser, encoder and heads from the source data; seed 42 fixes the Room B split, network initialisation and support draws. run_episode.py loads the frozen encoder, normaliser, router and default heads (support drawn from development captures at the default K) and can refit the heads from a support CSV (path, room, label); it writes window_id, room, label, confidence.

10. Contribution Statement

This project was completed individually by Harshit Singhal (23CS10025): data inspection, PCAP and Nexmon decoding pipeline, quality control and splitting, windowing and normalisation, the three tasks, evaluation, figures and this report.

11. AI-Use Disclosure

AI assistants were used for code generation: most of the Python source (ingestion, decoding, windowing, encoder, adaptation and evaluation scripts) was produced with AI help. Intermediate results were not taken on trust but checked by self-inspection, for example frame and packet counts and the Nexmon magic in the PCAPs, the position of the DC null across subcarriers, QC counters, capture and window counts against the specification, and the leakage assertions on splits and support sets. No source-author MATLAB code, processed .mat files or other teams' code were used. I reviewed the code and can explain each stage of the pipeline.

References

[1] Schäfer, J.; Barrsiwal, B.R.; Kokhkharova, M.; Adil, H.; Liebehenschel, J. Human Activity Recognition Using CSI Information with Nexmon. Applied Sciences 2021, 11(19), 8860. https://doi.org/10.3390/app11198860

[2] IEEE DataPort, CSI Human Activity dataset. https://ieee-dataport.org/open-access/csi-human-activity

[3] Nexmon CSI project, packed CSI format documentation. https://github.com/seemoo-lab/nexmon_csi

[4] Chen, T.; Kornblith, S.; Norouzi, M.; Hinton, G. A Simple Framework for Contrastive Learning of Visual Representations. ICML 2020.

[5] Snell, J.; Swersky, K.; Zemel, R. Prototypical Networks for Few-shot Learning. NeurIPS 2017.